

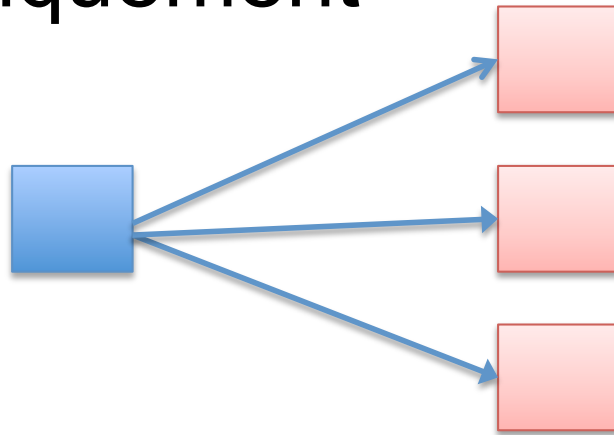
Le patron Observateur

Observer Pattern

Préparé par MAA S.Yahiouche

But

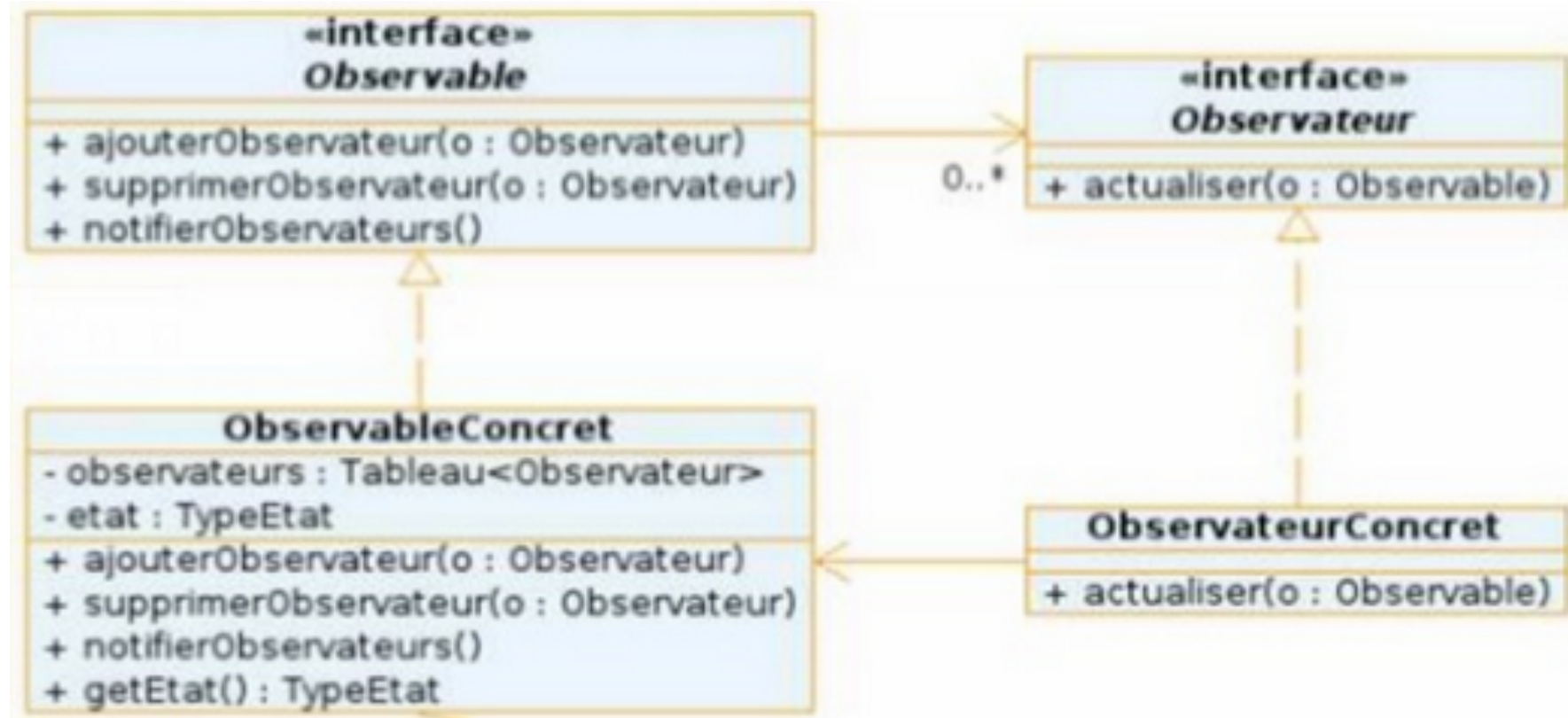
- Définit une relation de dépendance 1 à N entre les objets de telle manière que si un objet change d'état, tous les objets dépendants sont notifiés et mis à jour automatiquement



Fonctionnement

- Est utilisé pour renvoyer un signal à des modules qui jouent le rôle d'observateurs
- En cas de notification, les observateurs effectuent l'action adéquate en fonction des informations qui parviennent depuis les modules qu'ils observent
- Le principe est que chaque classe observable contient une liste d'observateurs, ainsi à l'aide d'une méthode de notification, l'ensemble des observateurs est prévenu

Diagramme UML



Participants

- "**Observable** » ou **Sujet** est une classe qui représente l'objet à observer, elle doit déclencher des événements, on ajoute:
 - En attribut : une liste d'Observateurs
 - Une méthode permettant d'ajouter un Observateur dans la liste
 - Une méthode permettant d'envoyer un signal à tous ses observeurs.
- "**Observateur**" est une classe abstraite avec une méthode actualiser, dont héritent des observateurs "**concrets**" qui implémentent cette méthode.
- Quand l'état de la classe change elle doit envoyer un signal à tout ses observateurs qui doivent effectuer l'action nécessaire en fonction du nouvel état de la classe.

Exemple: avertir quand une base de donnée est modifiée

- La classe observable est une base de donnée
- L'observateur peut être l'administrateur de la base de données, le chef de l'entreprise où la classe Archive (qui archive le journal log de la base de données)

Exemple: l'interface observable

```
public interface Subject {  
    public void registerObserver(Observer o);  
    public void removeObserver(Observer o);  
    public void notifyObservers();  
}
```

L'interface Observateur

```
public interface Observer {  
public void update(String operation, String  
    record);  
}
```

Exemple la classe concrète du sujet Observable

```
import java.util.*;

public class Database implements Subject {
    private List<Observer> observers;

    public Database() {
        observers = new ArrayList<Observer>(); }

    public void registerObserver(Observer o) {
        observers.add(o); }

    public void removeObserver(Observer o) {
        observers.remove(o); }

}
```

Ajouter la méthode qui déclenche l'évènement

- Dans notre exemple, on va la nommer editDB(); elle peut-t-être appelée (depuis une méthode main()) comme suit:
- `db.editDb(« delete »,24);`(db est un objet de la classe Database)
- Cette méthode aura 2 paramètres: l'opération et le numéro de l'enregistrement à modifier
- On doit donc rajouter deux attributs à la classe Database: opération et numRecord pour capter les deux paramètres de editDb(), afin de pouvoir les transmettre à l'observateur via la méthode notify(qu'on va implémenter plus loin)

Implémentation de la méthode Déclencheur d'évènement

```
import java.util.*;

public class Database implements Subject {
    private List<Observer> observers;

    private String operation;
    private int numRecord;
    .....//les autres méthodes

    public void editDb(String operation, int record)
    {
        this.operation = operation;
        numRecord = record;
        notifyObservers();
    }
}
```

Implémentation de la méthode notify()

```
import java.util.*;
```

```
public class Database implements Subject {  
    private List<Observer> observers;
```

```
    private String operation;
```

```
    private int numRecord;
```

```
    .....//les autres méthodes
```

```
    public void notifyObservers() {  
        for (Observer obs:observers)  
            observer.update(operation, record);  
    } }
```

```
    public void editDb(String operation, int record)  
    {  
        this.operation = operation;  
        numRrecord = record;  
        notifyObservers();  
    } }
```

Implémenter les observateur

- Comme mentionné au diapo 6, on peut avoir plusieurs classes d'observateurs, chacune doit implémenter l'interface Observer

Un premier observateur

```
public class Archiver implements Observer {  
    public Archiver() {  
        }  
    public void update(String operation, String  
        record) {  
        System.out.println("L'archiviste dit que  
            l'opération " + operation + " a été effectuée  
            sur l'enregistrement numéro " + record);  
        }  
    }  
}
```

Un deuxième observateur

```
public class Admin implements Observer {  
    public Admin() {  
        }  
    public void update(String operation, String  
        record) {  
        System.out.println("L'admin dit que l'opération  
            " + operation + " a été effectuée sur  
            l'enregistrement numéro " + record);  
        }  
    }  
}
```

Un troisième observateur

```
public class Boss implements Observer {  
    public Boss() {  
    }  
    public void update(String operation, String  
        record) {  
        System.out.println("Le directeur dit que  
            l'opération " + operation + " a été effectuée  
            sur l'enregistrement numéro " + record);  
    }  
}
```

Remarque

- On peut simplifier l'exemple on n'implémentant qu'une seule classe Observer concrète:

```
public class ObserverConcret implements Observer {
```

```
private String nomObs;
```

```
public ObserverConcret (String nomObs) {
```

```
    This.nomObs=nomObs  
}
```

```
public void update(String operation, String record) {
```

```
    System.out.println("Le "+nomObs+" dit que l'opération " + operation +  
        " a été effectuée sur l'enregistrement numéro " + record);
```

```
}}
```

Mais dans la réalité cela ne se passe pas aussi simplement; une Classe Admin est en fait totalement différente d'une classe Archive et d'une Classe Directeur

Tester les observateurs de la base de données

```
public class TestObserver {  
public static void main(String  
    args[]) {
```

//instancier de sujet
observable:

```
Database db = new  
    Database();
```

//instancier des observateurs:

```
Archiver archiver = new  
    Archiver();  
Client client = new Client();  
Boss boss = new Boss();
```

//attacher les observateurs au
sujet Observable:

```
db.registerObserver(archiver);  
db.registerObserver(client);  
db.registerObserver(boss);
```

//déclencher un événement sur
le sujet(bdd) en supprimant
par exemple
l'enregistrement N°24:

```
db.editDb("delete", 24);  
}  
}
```

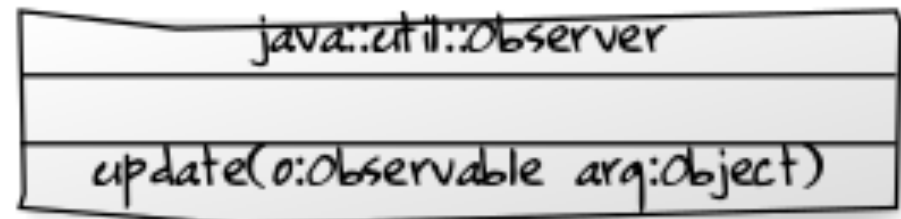
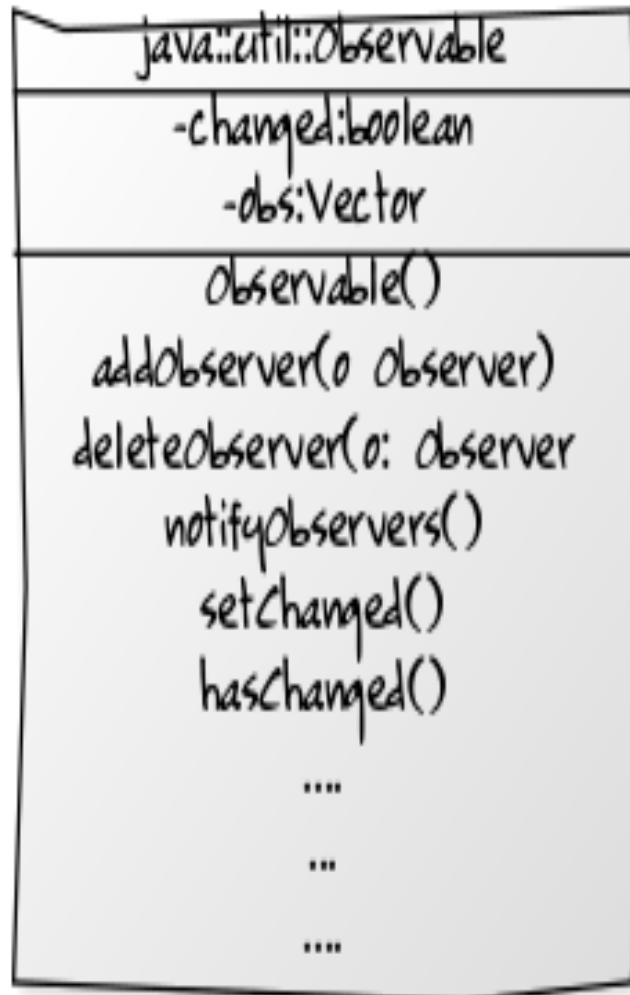
Compilation et exécution

- On aura sur la console les trois messages suivant:
- L'admin dit que l'opération delete a été effectuée sur l'enregistrement numéro 24
- L'archiviste dit que l'opération delete a été effectuée sur l'enregistrement numéro 24
- Le directeur dit que l'opération delete a été effectuée sur l'enregistrement numéro 24

Le patron Observer intégré dans Java

- Java prend en charge nativement dans plusieurs de ses API le patron Observer
- Le cas le plus général est constitué d'une interface Observer et d'une classe Observable du package java.util.
- Elles sont très similaires à nos interfaces, mais elles apportent d'autres fonctionnalités « toutes prêtes ».

La classe Observable et l'interface Observer de l'api Java



Observer dans l'api Java

- Observable est une CLASSE, non une interface. Elle mémorise tous les observateurs et leurs envoie des notifications à votre place
- L'interface Observer est la même que dans notre précédent diagramme de classes sauf que le nom de la méthode actualiser() est devenu update().
- Dans la classe ObservableConcret nous n'avons plus besoin d'implémenter les méthode ajouter(), supprimer() et notifier(), elles sont hérités de la classe Observable

Deux Règles pour déclencher un évènement

Si l'on utilise le patron observer de java, l'objet observé doit suivre deux règles

1. S'il est modifié, il doit appeler `setChanged()`
2. Quand il est prêt à notifier les observateurs, il appelle `notifyObservers()`

Notre exemple avec le patron Observer de java le sujet concret

```
Imports java.util.Observer;this.operation = operation;
public class Database      numRrecord = record;
    extends Observable {   setChanged();
                           notifyObservers();
                           }
private String operation;
private int numRecord;    public int getRecord() {
                           return record; }
                           public String
                           getOperation() {
                           return operation; }
                           }

public void editDb(String
    operation, int record)
{
}
```



Notre exemple avec le patron Observer de java

le 1^{er} Observateur

```
Imports java.util.Observer;
```

```
public class Archiver implements Observer {  
    public Archiver() {  
        } Observable  
    public void update(Observer obs, Object record) {  
        System.out.println("L'archivageur dit que l'opération " +  
            ((Database)obs).getOperation()  
            + " a été effectuée sur l'enregistrement numéro " +  
            ((Database)obs).getRecord());  
    }  
}
```

Notre exemple avec le patron Observer de java

le 2eme Observateur

```
Imports java.util.Observer;
```

```
public class Admin implements Observer {  
    public Admin() {  
        }  
        Observable  
    public void update(Observer obs, Object record) {  
        System.out.println("L'Admin dit que l'opération " +  
            ((Database)obs).getOperation()  
            + " a été effectuée sur l'enregistrement numéro " +  
            ((Database)obs).getRecord());  
    }  
}
```

Notre exemple avec le patron Observer de java

le 3eme Observateur

```
Imports java.util.Observer;
```

```
public class Boss implements Observer {  
    public Boss() {  
        }  
        Observable  
    public void update(Observer obs, Object record) {  
        System.out.println("Le directeur dit que l'opération " +  
            ((Database)obs).getOperation()  
            + " a été effectuée sur l'enregistrement numéro " +  
            ((Database)obs).getRecord());  
    }  
}
```

Notre exemple avec le patron Observer de java

la classe Test

```
public class TestObserver {  
public static void main(String  
    args[]) {
```

```
//instancier de sujet observable:  
Database db = new Database();
```

```
//instancier des observateurs:
```

```
Archiver archiver = new  
    Archiver();  
Client client = new Client();  
Boss boss = new Boss();
```

```
//attacher les observateurs au  
sujet Observable, avec la  
méthode héritée  
addObserver(o: Observer):
```

```
db.addObserver(archiver);  
db. addObserver(client);  
db. addObserver(boss);
```

```
//déclencher un événement sur le  
sujet(bdd) en supprimant par  
exemple l'enregistrement N°24:
```

```
db.editDb("delete", 24);  
}  
}
```

Remarque

- Un petit inconvénient du patron Observer implémenté dans java:
 - Si notre classe SujetConcret hérite auparavant d'une classe essentielle pour son fonctionnement, elle ne peut pas étendre Observable de Java, car le multi Héritage est interdit en Java
- Donc, pour une application robuste et extensible, il est préférable d'utiliser le patron Observer général plutôt que celui implémenté dans java

Observer et les ActionListener de java

- Le composant Swing JButton est un sujet qui hérite de la classe AbstractButton.
- Si vous regardez la structure de cette classe vous trouverez des méthodes comme add et remove qui permettent d'ajouter et de supprimer des observateurs qu'on appelle dans la terminologie de Swing des listeners (Auditeurs).

Observer et les ActionListener de java

- Ces listeners sont à l'écoute des différents types d'événements qui peuvent se produire dans le composant JButton.
- Par exemple un ActionListener vous permet d'écouter tous les types d'actions qui peuvent être appliquées à un bouton, comme les clics.
- Si l'un des événements se produit sur le bouton, ce dernier notifie tous les observateurs qui sont de type ActionListener en faisant appel à la méthode actionPerformed() de chaque ActionListener.

Exemple avec ActionListener

```
import java.awt.BorderLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JTextArea;
```

```
public class SimpleSwingExample {
```

```
    public static void main(String[] args) {
        JFrame frame = new JFrame("Frame Title");
        final JTextArea comp = new JTextArea();
        JButton btn = new JButton("click");
        frame.getContentPane().add(comp, BorderLayout.CENTER);
        frame.getContentPane().add(btn, BorderLayout.SOUTH);
```

```
        btn.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent ae) {
                comp.setText("Button has been clicked");
            }
        });
```

```
        int width = 300;
        int height = 300;
        frame.setSize(width, height);
```

```
        frame.setVisible(true);
```

```
    }
```

```
}
```

Enregistrement
du bouton avec
les
observateurs(List
eners)

